

1. Tumoral dynamics

Contents

1	Environment preparation	2
2	Data formatting	2
3	Survival object	3
4	Proportional hazards assumption check : Schoenfeld residuals	3
5	AIC comparison of different distributions	4
6	Model selection	4
6.1	Random effects selection	4
6.2	Fixed effects selection	5
6.3	Goodness-of-fit evaluation	5
6.4	Selected model summary	6
7	Median times	8
7.1	By sleep condition	8
7.2	By lineage	9
7.3	By sleep condition and lineage	9
8	Cumulative incidences	10
8.1	By week and sleep condition	10
8.2	By week and lineage	11
8.3	By week, sleep condition, and lineage	11
9	Cumulative proportions	13
9.1	By week and sleep condition	13
9.2	By week and lineage	13
9.3	By week, sleep condition, and lineage	14

10 Plots of cumulative incidence of tumoral development	16
10.1 Common themes	16
10.2 Panel A: by sleep condition	17
10.3 Panel B: by sleep condition for each lineage	17
10.3.1 HO_MT	17
10.3.2 HO_SPC	18
10.3.3 HO_SPT	18
10.4 Panel C: by lineage	18
10.4.1 HO_MT	18
10.4.2 HO_SPC	19
10.4.3 HO_SPT	19
10.5 Assembly	19

1 Environment preparation

Run scripts with the needed packages, functions, and the raw data frame.

```
source("0a_packages.R")
source("0b_script_data.R")
source("0c_functions.R")
```

2 Data formatting

Create a new data frame adapted for survival analysis.

```
tum_df <-

# Load the raw data frame.
df %>%

# Restrict the data frame to the three lineages that developed tumors.
filter(lineage %in% c("HO_MT", "HO_SPC", "HO_SPT")) %>%

# Define the follow-up time for each individual:
# if the hydra developed a tumor -> time of tumor onset (event),
# if the hydra died -> time of death (censor),
# if the hydra did not develop a tumor and stayed alive
# -> 42 days (no event, censor at the end of the experiment).
mutate(age_followup_days = case_when(
  tum_state == "Tumoral" ~ age_tum,
  tum_state == "Healthy" & death_state == 1 ~ age_death,
  TRUE ~ 42),

age_followup = age_followup_days / 7,
tum_event = ifelse(tum_state == "Tumoral", 1, 0)) %>%
```

```

#' Build a competing-risk variable for the cumulative incidence functions:
#' if the hydra did not develop a tumor and stayed alive -> 0 (no event),
#' if the hydra developed a tumor -> 1 (event),
#' if the hydra died -> 2 (competing risk of death).
mutate(status_competing = factor(case_when(tum_event == 1 ~ 1,
                                           death_state == 1 ~ 2,
                                           TRUE ~ 0))) %>%

#' Remove missing data.
filter(!is.na(age_followup)) %>%

#' Set factor levels for later use in the plots.
mutate(sleep_condition = factor(sleep_condition,
                                levels = c("Control", "Mechanical sleep disruption")))
%>%

#' Keep only the variables needed for the analysis.
select(age_followup, tum_event, status_competing, sleep_condition,
       lineage, parent_unique, replicate_unique)

```

Set reference levels for the categorical variables.

```

tum_df$lineage <- relevel(factor(tum_df$lineage), ref = "HO_MT")
tum_df$sleep_condition <- relevel(factor(tum_df$sleep_condition), ref = "Control")

```

3 Survival object

Create a survival object with follow-up time and event indicator.

```
tum_surv_object <- Surv(tum_df$age_followup, tum_df$tum_event)
```

4 Proportional hazards assumption check : Schoenfeld residuals

Fit a preliminary full Cox model (fixed effects only).

```
tum_cox <- coxph(Surv(age_followup, tum_event) ~ sleep_condition * lineage, data =
tum_df)
```

Test the proportional hazards assumption.

```
print(cox.zph(tum_cox))
```

```
##                chisq df      p
## sleep_condition  0.575  1 0.448
## lineage         5.279  2 0.071
## sleep_condition:lineage 8.986  2 0.011
## GLOBAL         11.395  5 0.044
```

The proportional hazards assumption is violated for at least one term, an accelerated failure time mixed-effects model will be used.

5 AIC comparison of different distributions

Compare intercept-only accelerated failure time models fitted with different parametric distributions to identify the best-fitting one.

```
dist_weib <- SurvregME(Surv(age_followup, tum_event) ~ 1, data = tum_df,
  dist = "weibull")

dist_lnorm <- SurvregME(Surv(age_followup, tum_event) ~ 1, data = tum_df,
  dist = "lognormal")

dist_llog <- SurvregME(Surv(age_followup, tum_event) ~ 1, data = tum_df,
  dist = "loglogistic")

dist_exp <- SurvregME(Surv(age_followup, tum_event) ~ 1, data = tum_df,
  dist = "exponential")

AIC(dist_weib, dist_lnorm, dist_llog, dist_exp) %>% arrange(AIC)
```

```
##      df      AIC
## dist_weib  2 1291.277
## dist_llog  2 1294.009
## dist_lnorm 2 1334.860
## dist_exp   1 1347.392
```

The model with a Weibull distribution has the lowest AIC, we'll use it for the models.

6 Model selection

6.1 Random effects selection

With the full fixed-effect structure held constant, compare different random-effect structures to find the optimal one.

```
tum_R_0 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage,
  data = tum_df, dist = "weibull")

tum_R_1 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage +
  (1 | parent_unique),
  data = tum_df, dist = "weibull")

tum_R_2 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage +
  (1 | replicate_unique),
  data = tum_df, dist = "weibull")

tum_R_3 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage +
  (1 | parent_unique) + (1 | replicate_unique),
```

```
data = tum_df, dist = "weibull")

AIC(tum_R_0, tum_R_1, tum_R_2, tum_R_3) %>% arrange(AIC)
```

```
##      df      AIC
## tum_R_0  7 1157.217
## tum_R_2  8 1158.027
## tum_R_1  8 1159.087
## tum_R_3  9 1160.027
```

The model without any random effect (tum_R_0) has the lowest AIC, this random effect structure is therefore selected. SurvregME is nonetheless kept as the modeling function (rather than switching to a simpler survreg call) since it also supports models with no random effects.

6.2 Fixed effects selection

With the selected random-effect structure fixed, compare different fixed-effect structures to find the optimal one.

```
tum_F_0 <- SurvregME(Surv(age_followup, tum_event) ~ 1,
  data = tum_df, dist = "weibull")

tum_F_1 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition,
  data = tum_df, dist = "weibull")

tum_F_2 <- SurvregME(Surv(age_followup, tum_event) ~ lineage,
  data = tum_df, dist = "weibull")

tum_F_3 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition + lineage,
  data = tum_df, dist = "weibull")

tum_F_4 <- SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage,
  data = tum_df, dist = "weibull")

AIC(tum_F_0, tum_F_1, tum_F_2, tum_F_3, tum_F_4) %>% arrange(AIC)
```

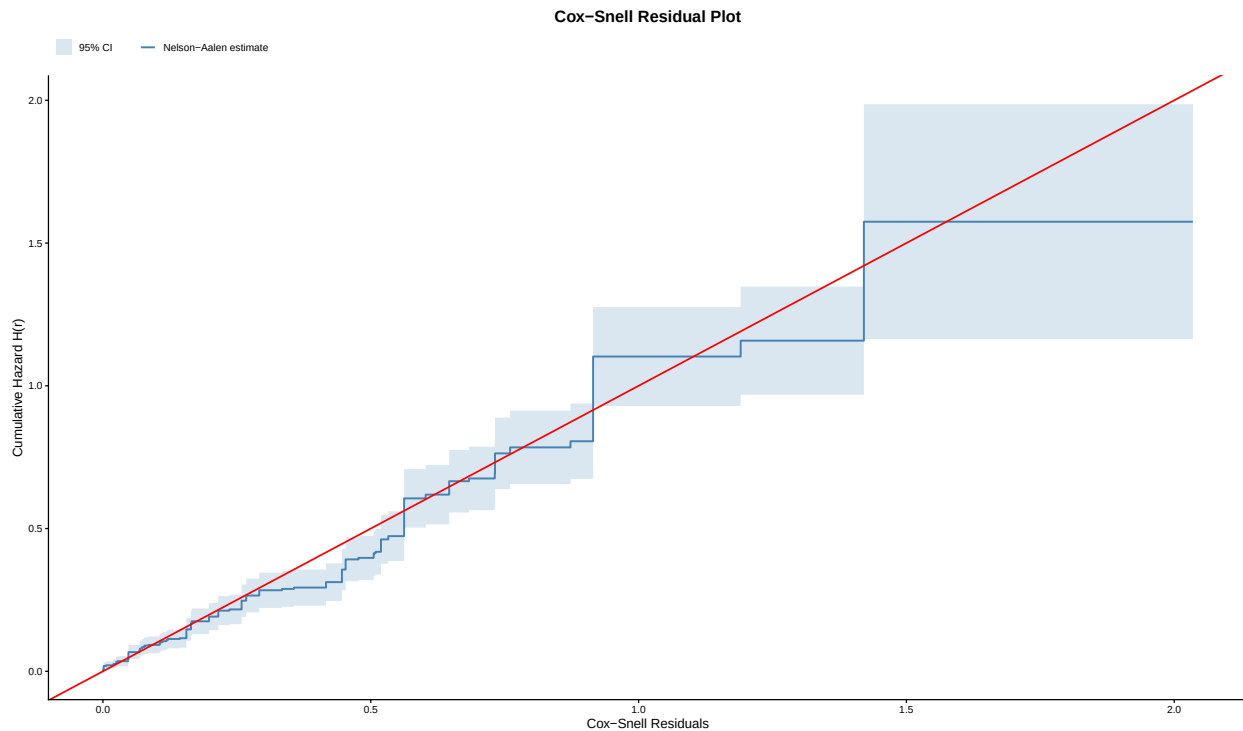
```
##      df      AIC
## tum_F_4  7 1157.217
## tum_F_3  5 1203.737
## tum_F_2  4 1212.049
## tum_F_1  3 1275.994
## tum_F_0  2 1291.277
```

The model with the interaction between the sleep condition and the lineage has the lowest AIC and is therefore selected (tum_F_4).

6.3 Goodness-of-fit evaluation

Assess goodness-of-fit of the selected model using Cox-Snell residuals.

```
plot_cox_snell_survregme(model = tum_F_4, data = tum_df,
                        time_col = "age_followup", event_col = "tum_event")
```



The curve follows a 45 degree exponential, we validate the fit of the selected model.

6.4 Selected model summary

Releveling of “lineage” to obtain all pairwise comparisons from the same selected model.

```
tum_df$lineage <- releve(tum_df$lineage, ref = "HO_MT")
summary(SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage,
                  data = tum_df, dist = "weibull"))
```

```
##
## Weibull Linear Regression Model
##
## Formula:
## Surv(age_followup, tum_event) ~ sleep_condition * lineage
##
## Fitted to dataset tum_df
##
## Fixed effects parameters:
## =====
##
##
```

	Estimate	Std. Error
## sleep_conditionMechanical sleep disruption	-1.439789	0.359848
## lineageHO_SPC	0.016721	0.447214
## lineageHO_SPT	-2.582680	0.343763

```
## sleep_conditionMechanical sleep disruption:lineageHO_SPC -0.372221 0.502700
## sleep_conditionMechanical sleep disruption:lineageHO_SPT 1.926839 0.409974
## z value Pr(>|z|)
## sleep_conditionMechanical sleep disruption -4.0011 6.305e-05 ***
## lineageHO_SPC 0.0374 0.9702
## lineageHO_SPT -7.5130 5.780e-14 ***
## sleep_conditionMechanical sleep disruption:lineageHO_SPC -0.7404 0.4590
## sleep_conditionMechanical sleep disruption:lineageHO_SPT 4.6999 2.603e-06 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log-likelihood: -571.6086 (npar = 7)
```

```
tum_df$lineage <- relevel(tum_df$lineage, ref = "HO_SPC")
summary(SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage,
  data = tum_df, dist = "weibull"))
```

```
##
## Weibull Linear Regression Model
##
## Formula:
## Surv(age_followup, tum_event) ~ sleep_condition * lineage
##
## Fitted to dataset tum_df
##
## Fixed effects parameters:
## =====
##
## Estimate Std. Error
## sleep_conditionMechanical sleep disruption -1.812001 0.351408
## lineageHO_MT -0.016705 0.447214
## lineageHO_SPT -2.599394 0.343686
## sleep_conditionMechanical sleep disruption:lineageHO_MT 0.372206 0.502700
## sleep_conditionMechanical sleep disruption:lineageHO_SPT 2.299055 0.402866
## z value Pr(>|z|)
## sleep_conditionMechanical sleep disruption -5.1564 2.517e-07 ***
## lineageHO_MT -0.0374 0.9702
## lineageHO_SPT -7.5633 3.930e-14 ***
## sleep_conditionMechanical sleep disruption:lineageHO_MT 0.7404 0.4590
## sleep_conditionMechanical sleep disruption:lineageHO_SPT 5.7067 1.152e-08 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log-likelihood: -571.6086 (npar = 7)
```

```
tum_df$lineage <- relevel(tum_df$lineage, ref = "HO_SPT")
summary(SurvregME(Surv(age_followup, tum_event) ~ sleep_condition * lineage,
  data = tum_df, dist = "weibull"))
```

```
##
## Weibull Linear Regression Model
##
```

```
## Formula:
## Surv(age_followup, tum_event) ~ sleep_condition * lineage
##
## Fitted to dataset tum_df
##
## Fixed effects parameters:
## =====
##
##                                     Estimate Std. Error
## sleep_conditionMechanical sleep disruption      0.48705      0.19562
## lineageHO_SPC                                     2.59940      0.34369
## lineageHO_MT                                     2.58269      0.34376
## sleep_conditionMechanical sleep disruption:lineageHO_SPC -2.29906      0.40287
## sleep_conditionMechanical sleep disruption:lineageHO_MT -1.92685      0.40997
##                                     z value  Pr(>|z|)
## sleep_conditionMechanical sleep disruption      2.4898    0.01278 *
## lineageHO_SPC                                     7.5633 3.930e-14 ***
## lineageHO_MT                                     7.5130 5.780e-14 ***
## sleep_conditionMechanical sleep disruption:lineageHO_SPC -5.7068 1.152e-08 ***
## sleep_conditionMechanical sleep disruption:lineageHO_MT -4.6999 2.603e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log-likelihood: -571.6086 (npar = 7)
```

7 Median times

7.1 By sleep condition

Estimate cumulative incidence functions for the tumor event, accounting for the competing risk of death, stratified by sleep condition.

```
tum_cuminc_sleep <- cuminc(Surv(age_followup, status_competing) ~
                           sleep_condition, data = tum_df)
```

For each stratum, extract the median time to event and its 95CI.

```
tum_medians_sleep <- tidy(tum_cuminc_sleep) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::group_by(strata) %>%
  dplyr::arrange(time) %>%
  dplyr::summarise(
    median_time = { idx <- which(estimate >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_low = { idx <- which(conf.high >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_high = { idx <- which(conf.low >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    .groups = "drop")

print(tum_medians_sleep)
```

```
## # A tibble: 2 x 4
##   strata                median_time ci_low ci_high
##   <fct>                <dbl>   <dbl>   <dbl>
## 1 Control                NA     NA       NA
## 2 Mechanical sleep disruption    5   4.14   5.43
```

7.2 By lineage

Same method as above, but stratified by lineage.

```
tum_cuminc_lineage <- cuminc(Surv(age_followup, status_competing) ~ lineage, data =
tum_df)

tum_medians_lineage <- tidy(tum_cuminc_lineage) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::group_by(strata) %>%
  dplyr::arrange(time) %>%
  dplyr::summarise(
    median_time = { idx <- which(estimate >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_low = { idx <- which(conf.high >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_high = { idx <- which(conf.low >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    .groups = "drop")

print(tum_medians_lineage)
```

```
## # A tibble: 3 x 4
##   strata median_time ci_low ci_high
##   <fct>      <dbl>   <dbl>   <dbl>
## 1 HO_SPT         4       4       4
## 2 HO_SPC        NA      NA      NA
## 3 HO_MT         NA      NA      NA
```

7.3 By sleep condition and lineage

Same method as above, but stratified by sleep condition and lineage.

```
tum_cuminc_interaction <- cuminc(Surv(age_followup, status_competing) ~
sleep_condition + lineage, data = tum_df)

tum_medians_interaction <- tidy(tum_cuminc_interaction) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::group_by(strata) %>%
  dplyr::arrange(time) %>%
  dplyr::summarise(
    median_time = { idx <- which(estimate >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_low = { idx <- which(conf.high >= 0.5)[1]
    if (is.na(idx)) NA_real_ else time[idx] },
    ci_high = { idx <- which(conf.low >= 0.5)[1]
```

```

    if (is.na(idx)) NA_real_ else time[idx] },
    .groups = "drop")

print(tum_medians_interaction)

```

```

## # A tibble: 6 x 4
##   strata                                median_time ci_low ci_high
##   <fct>                                <dbl>   <dbl>   <dbl>
## 1 Control, HO_SPT                      4     4       4
## 2 Mechanical sleep disruption, HO_SPT  4     4     4.57
## 3 Control, HO_SPC                      NA    NA      NA
## 4 Mechanical sleep disruption, HO_SPC  5     4.14   NA
## 5 Control, HO_MT                      NA    NA      NA
## 6 Mechanical sleep disruption, HO_MT  NA     5      NA

```

8 Cumulative incidences

8.1 By week and sleep condition

Convert the continuous cumulative incidence function into a weekly summary table: for each stratum of sleep condition and each week, show the estimate with its 95CI.

```

tum_incidence_sleep <- tidy(tum_cuminc_sleep) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::mutate(week = ceiling(time)) %>%
  dplyr::group_by(strata, week) %>%
  dplyr::filter(time == max(time)) %>%
  dplyr::slice(1) %>%
  dplyr::ungroup() %>%
  dplyr::select(strata, week, estimate_cuminc = estimate, conf.low, conf.high) %>%
  dplyr::arrange(strata, week)

print(tum_incidence_sleep)

```

```

## # A tibble: 14 x 5
##   strata                                week estimate_cuminc conf.low conf.high
##   <fct>                                <dbl>         <dbl>   <dbl>   <dbl>
## 1 Control                                0           0      NA      NA
## 2 Control                                1         0.0278  0.0115  0.0565
## 3 Control                                2         0.0417  0.0205  0.0745
## 4 Control                                3         0.102   0.0660  0.147
## 5 Control                                4         0.296   0.237   0.358
## 6 Control                                5         0.356   0.293   0.420
## 7 Control                                6         0.370   0.306   0.435
## 8 Mechanical sleep disruption            0           0      NA      NA
## 9 Mechanical sleep disruption            1         0.0185  0.00615 0.0439
## 10 Mechanical sleep disruption           2         0.0833  0.0513  0.125
## 11 Mechanical sleep disruption           3         0.167   0.120   0.219
## 12 Mechanical sleep disruption           4         0.384   0.319   0.449
## 13 Mechanical sleep disruption           5         0.537   0.468   0.601
## 14 Mechanical sleep disruption           6         0.574   0.505   0.637

```

8.2 By week and lineage

Same method as above, but stratified by lineage.

```
tum_incidence_lineage <- tidy(tum_cuminc_lineage) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::mutate(week = ceiling(time)) %>%
  dplyr::group_by(strata, week) %>%
  dplyr::filter(time == max(time)) %>%
  dplyr::slice(1) %>%
  dplyr::ungroup() %>%
  dplyr::select(strata, week, estimate_cuminc = estimate, conf.low, conf.high) %>%
  dplyr::arrange(strata, week)

print(tum_incidence_lineage)
```

```
## # A tibble: 21 x 5
##   strata week estimate_cuminc conf.low conf.high
##   <fct> <dbl>         <dbl>     <dbl>     <dbl>
## 1 HO_SPT  0           0         NA         NA
## 2 HO_SPT  1       0.0486    0.0214    0.0926
## 3 HO_SPT  2       0.139     0.0883    0.201
## 4 HO_SPT  3       0.257     0.189     0.331
## 5 HO_SPT  4       0.653     0.569     0.725
## 6 HO_SPT  5       0.743     0.663     0.807
## 7 HO_SPT  6       0.743     0.663     0.807
## 8 HO_SPC  0           0         NA         NA
## 9 HO_SPC  1     0.00694    0.000628  0.0351
##10 HO_SPC  2     0.00694    0.000628  0.0351
##11 HO_SPC  3     0.0833     0.0454    0.136
##12 HO_SPC  4     0.181     0.123     0.248
##13 HO_SPC  5     0.333     0.257     0.411
##14 HO_SPC  6     0.368     0.290     0.447
##15 HO_MT   0           0         NA         NA
##16 HO_MT   1     0.0139    0.00272   0.0452
##17 HO_MT   2     0.0417    0.0171   0.0836
##18 HO_MT   3     0.0625    0.0306   0.110
##19 HO_MT   4     0.188     0.128     0.255
##20 HO_MT   5     0.264     0.195     0.338
##21 HO_MT   6     0.306     0.232     0.382
```

8.3 By week, sleep condition, and lineage

Same method as above, but stratified by sleep condition and lineage.

```
tum_incidence_interaction <- tidy(tum_cuminc_interaction) %>%
  dplyr::filter(outcome == "1") %>%
  dplyr::mutate(week = ceiling(time)) %>%
  dplyr::group_by(strata, week) %>%
  dplyr::filter(time == max(time)) %>%
  dplyr::slice(1) %>%
  dplyr::ungroup() %>%
```

```
dplyr::select(strata, week, estimate_cuminc = estimate, conf.low, conf.high) %>%
dplyr::arrange(strata, week)

print(tum_incidence_interaction)
```

```
## # A tibble: 42 x 5
##   strata                                week estimate_cuminc conf.low conf.high
##   <fct>                                <dbl>         <dbl>      <dbl>      <dbl>
## 1 Control, HO_SPT                      0             0         NA         NA
## 2 Control, HO_SPT                      1             0.0694    0.0255    0.144
## 3 Control, HO_SPT                      2             0.0972    0.0425    0.179
## 4 Control, HO_SPT                      3             0.264     0.168     0.370
## 5 Control, HO_SPT                      4             0.736     0.617     0.824
## 6 Control, HO_SPT                      5             0.833     0.722     0.903
## 7 Control, HO_SPT                      6             0.833     0.722     0.903
## 8 Mechanical sleep disruption, HO_SPT 0             0         NA         NA
## 9 Mechanical sleep disruption, HO_SPT 1             0.0278    0.00519   0.0871
## 10 Mechanical sleep disruption, HO_SPT 2             0.181     0.102     0.278
## 11 Mechanical sleep disruption, HO_SPT 3             0.25      0.157     0.355
## 12 Mechanical sleep disruption, HO_SPT 4             0.569     0.446     0.675
## 13 Mechanical sleep disruption, HO_SPT 5             0.653     0.529     0.751
## 14 Mechanical sleep disruption, HO_SPT 6             0.653     0.529     0.751
## 15 Control, HO_SPC                     0             0         NA         NA
## 16 Control, HO_SPC                     1             0         NA         NA
## 17 Control, HO_SPC                     2             0         NA         NA
## 18 Control, HO_SPC                     3             0.0139    0.00116   0.0669
## 19 Control, HO_SPC                     4             0.0556    0.0178    0.126
## 20 Control, HO_SPC                     5             0.111     0.0516    0.196
## 21 Control, HO_SPC                     6             0.139     0.0708    0.229
## 22 Mechanical sleep disruption, HO_SPC 0             0         NA         NA
## 23 Mechanical sleep disruption, HO_SPC 1             0.0139    0.00116   0.0669
## 24 Mechanical sleep disruption, HO_SPC 2             0.0139    0.00116   0.0669
## 25 Mechanical sleep disruption, HO_SPC 3             0.153     0.0809    0.246
## 26 Mechanical sleep disruption, HO_SPC 4             0.306     0.203     0.414
## 27 Mechanical sleep disruption, HO_SPC 5             0.556     0.432     0.662
## 28 Mechanical sleep disruption, HO_SPC 6             0.597     0.473     0.701
## 29 Control, HO_MT                      0             0         NA         NA
## 30 Control, HO_MT                      1             0.0139    0.00116   0.0669
## 31 Control, HO_MT                      2             0.0278    0.00519   0.0871
## 32 Control, HO_MT                      3             0.0278    0.00519   0.0871
## 33 Control, HO_MT                      4             0.0972    0.0424    0.179
## 34 Control, HO_MT                      5             0.125     0.0611    0.213
## 35 Control, HO_MT                      6             0.139     0.0708    0.229
## 36 Mechanical sleep disruption, HO_MT 0             0         NA         NA
## 37 Mechanical sleep disruption, HO_MT 1             0.0139    0.00116   0.0669
## 38 Mechanical sleep disruption, HO_MT 2             0.0556    0.0178    0.126
## 39 Mechanical sleep disruption, HO_MT 3             0.0972    0.0424    0.179
## 40 Mechanical sleep disruption, HO_MT 4             0.278     0.180     0.385
## 41 Mechanical sleep disruption, HO_MT 5             0.403     0.289     0.514
## 42 Mechanical sleep disruption, HO_MT 6             0.472     0.353     0.583
```

9 Cumulative proportions

9.1 By week and sleep condition

Compute raw cumulative proportions of tumor occurrence per week, stratified by sleep condition.

```
tum_proportions_sleep <- tum_df %>%
  dplyr::mutate(week_tum_event = ceiling(age_followup)) %>%
  dplyr::group_by(sleep_condition) %>%
  dplyr::summarise(total_N = n(),
    n1 = sum(week_tum_event <= 1 & tum_event == 1),
    n2 = sum(week_tum_event <= 2 & tum_event == 1),
    n3 = sum(week_tum_event <= 3 & tum_event == 1),
    n4 = sum(week_tum_event <= 4 & tum_event == 1),
    n5 = sum(week_tum_event <= 5 & tum_event == 1),
    n6 = sum(week_tum_event <= 6 & tum_event == 1),
    p1 = n1 / total_N,
    p2 = n2 / total_N,
    p3 = n3 / total_N,
    p4 = n4 / total_N,
    p5 = n5 / total_N,
    p6 = n6 / total_N,
    .groups = "drop") %>%
  tidyr::pivot_longer(cols = matches("^np[1-6]$"),
    names_to = c(".value", "week"),
    names_pattern = "(n|p)([1-6])") %>%
  dplyr::mutate(week = as.numeric(week)) %>%
  dplyr::select(sleep_condition, week, n, total_N, raw_proportion = p)

print(tum_proportions_sleep)
```

```
## # A tibble: 12 x 5
##   sleep_condition      week      n total_N raw_proportion
##   <fct>             <dbl> <int>   <int>         <dbl>
## 1 Control              1      6    216         0.0278
## 2 Control              2      9    216         0.0417
## 3 Control              3     22    216         0.102
## 4 Control              4     64    216         0.296
## 5 Control              5     77    216         0.356
## 6 Control              6     80    216         0.370
## 7 Mechanical sleep disruption 1      4    216         0.0185
## 8 Mechanical sleep disruption 2     18    216         0.0833
## 9 Mechanical sleep disruption 3     36    216         0.167
## 10 Mechanical sleep disruption 4     83    216         0.384
## 11 Mechanical sleep disruption 5    116    216         0.537
## 12 Mechanical sleep disruption 6    124    216         0.574
```

9.2 By week and lineage

Same method as above, but stratified by lineage.

```

tum_proportions_lineage <- tum_df %>%
  dplyr::mutate(week_tum_event = ceiling(age_followup)) %>%
  dplyr::group_by(lineage) %>%
  dplyr::summarise(total_N = n(),
    n1 = sum(week_tum_event <= 1 & tum_event == 1),
    n2 = sum(week_tum_event <= 2 & tum_event == 1),
    n3 = sum(week_tum_event <= 3 & tum_event == 1),
    n4 = sum(week_tum_event <= 4 & tum_event == 1),
    n5 = sum(week_tum_event <= 5 & tum_event == 1),
    n6 = sum(week_tum_event <= 6 & tum_event == 1),
    p1 = n1 / total_N,
    p2 = n2 / total_N,
    p3 = n3 / total_N,
    p4 = n4 / total_N,
    p5 = n5 / total_N,
    p6 = n6 / total_N,
    .groups = "drop") %>%
  tidyr::pivot_longer(cols = matches("^np[1-6]$"),
    names_to = c(".value", "week"),
    names_pattern = "(n|p)([1-6])" %>%
  dplyr::mutate(week = as.numeric(week)) %>%
  dplyr::select(lineage, week, n, total_N, raw_proportion = p)

print(tum_proportions_lineage)

```

```

## # A tibble: 18 x 5
##   lineage week      n total_N raw_proportion
##   <fct>   <dbl> <int>   <int>         <dbl>
## 1 HO_SPT     1      7    144         0.0486
## 2 HO_SPT     2     20    144         0.139
## 3 HO_SPT     3     37    144         0.257
## 4 HO_SPT     4     94    144         0.653
## 5 HO_SPT     5    107    144         0.743
## 6 HO_SPT     6    107    144         0.743
## 7 HO_SPC     1      1    144         0.00694
## 8 HO_SPC     2      1    144         0.00694
## 9 HO_SPC     3     12    144         0.0833
## 10 HO_SPC    4     26    144         0.181
## 11 HO_SPC    5     48    144         0.333
## 12 HO_SPC    6     53    144         0.368
## 13 HO_MT     1      2    144         0.0139
## 14 HO_MT     2      6    144         0.0417
## 15 HO_MT     3      9    144         0.0625
## 16 HO_MT     4     27    144         0.188
## 17 HO_MT     5     38    144         0.264
## 18 HO_MT     6     44    144         0.306

```

9.3 By week, sleep condition, and lineage

Same method as above, but stratified by sleep condition and lineage.

```

tum_proportions_interaction <- tum_df %>%
  dplyr::mutate(week_tum_event = ceiling(age_followup)) %>%
  dplyr::group_by(sleep_condition, lineage) %>%
  dplyr::summarise(total_N = n(),
    n1 = sum(week_tum_event <= 1 & tum_event == 1),
    n2 = sum(week_tum_event <= 2 & tum_event == 1),
    n3 = sum(week_tum_event <= 3 & tum_event == 1),
    n4 = sum(week_tum_event <= 4 & tum_event == 1),
    n5 = sum(week_tum_event <= 5 & tum_event == 1),
    n6 = sum(week_tum_event <= 6 & tum_event == 1),
    p1 = n1 / total_N,
    p2 = n2 / total_N,
    p3 = n3 / total_N,
    p4 = n4 / total_N,
    p5 = n5 / total_N,
    p6 = n6 / total_N,
    .groups = "drop") %>%
  tidyr::pivot_longer(cols = matches("^np[1-6]$"),
    names_to = c(".value", "week"),
    names_pattern = "(n|p)([1-6])" %>%
  dplyr::mutate(week = as.numeric(week)) %>%
  dplyr::select(sleep_condition, lineage, week, n, total_N, raw_proportion = p)

print(tum_proportions_interaction)

```

```

## # A tibble: 36 x 6
##   sleep_condition      lineage week    n total_N raw_proportion
##   <fct>              <fct>  <dbl> <int>  <int>      <dbl>
## 1 Control            HO_SPT    1     5    72      0.0694
## 2 Control            HO_SPT    2     7    72      0.0972
## 3 Control            HO_SPT    3    19    72      0.264
## 4 Control            HO_SPT    4    53    72      0.736
## 5 Control            HO_SPT    5    60    72      0.833
## 6 Control            HO_SPT    6    60    72      0.833
## 7 Control            HO_SPC    1     0    72      0
## 8 Control            HO_SPC    2     0    72      0
## 9 Control            HO_SPC    3     1    72      0.0139
## 10 Control           HO_SPC    4     4    72      0.0556
## 11 Control           HO_SPC    5     8    72      0.111
## 12 Control           HO_SPC    6    10    72      0.139
## 13 Control           HO_MT     1     1    72      0.0139
## 14 Control           HO_MT     2     2    72      0.0278
## 15 Control           HO_MT     3     2    72      0.0278
## 16 Control           HO_MT     4     7    72      0.0972
## 17 Control           HO_MT     5     9    72      0.125
## 18 Control           HO_MT     6    10    72      0.139
## 19 Mechanical sleep disruption HO_SPT    1     2    72      0.0278
## 20 Mechanical sleep disruption HO_SPT    2    13    72      0.181
## 21 Mechanical sleep disruption HO_SPT    3    18    72      0.25
## 22 Mechanical sleep disruption HO_SPT    4    41    72      0.569
## 23 Mechanical sleep disruption HO_SPT    5    47    72      0.653
## 24 Mechanical sleep disruption HO_SPT    6    47    72      0.653
## 25 Mechanical sleep disruption HO_SPC    1     1    72      0.0139
## 26 Mechanical sleep disruption HO_SPC    2     1    72      0.0139

```

## 27	Mechanical	sleep disruption	HO_SPC	3	11	72	0.153
## 28	Mechanical	sleep disruption	HO_SPC	4	22	72	0.306
## 29	Mechanical	sleep disruption	HO_SPC	5	40	72	0.556
## 30	Mechanical	sleep disruption	HO_SPC	6	43	72	0.597
## 31	Mechanical	sleep disruption	HO_MT	1	1	72	0.0139
## 32	Mechanical	sleep disruption	HO_MT	2	4	72	0.0556
## 33	Mechanical	sleep disruption	HO_MT	3	7	72	0.0972
## 34	Mechanical	sleep disruption	HO_MT	4	20	72	0.278
## 35	Mechanical	sleep disruption	HO_MT	5	29	72	0.403
## 36	Mechanical	sleep disruption	HO_MT	6	34	72	0.472

10 Plots of cumulative incidence of tumoral development

10.1 Common themes

Theme of y-axis, and its presence or absence on the plots

```
scale_y <- scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, by = 0.25),
                              labels = percent, position = "right")
```

Fix to hide y-axis numbers, ticks, and titles on the right and left

```
hide_y <- theme(axis.text.y.right = element_blank(),
                 axis.ticks.y.right = element_blank(),
                 axis.title.y.right = element_blank(),
                 axis.text.y = element_blank(),
                 axis.ticks.y = element_blank(),
                 axis.title.y = element_blank())
```

Fix to show y-axis numbers and ticks on the right

```
show_y <- theme(axis.text.y.right = element_text(color = "black", size = 14),
                 axis.ticks.y.right = element_line(),
                 axis.text.y = element_blank(),
                 axis.ticks.y = element_blank(),
                 axis.title.y = element_blank())
```

Theme of x-axis

```
scale_x <- scale_x_continuous(limits = c(0, 6), breaks = seq(0, 6, by = 2))
```

Color scales for sleep condition

```
sleep_scale <- list(
  scale_color_manual(values = sleep_condition_colors, breaks = c("Control", "Mechanical
sleep disruption")),
  scale_fill_manual(values = sleep_condition_colors, breaks = c("Control", "Mechanical
sleep disruption")))
```

Theme shared by all panels (angle = 90 for bottom-to-top reading)

```
theme_common <- theme(
  axis.text.x = element_text(color = "black", size = 14),
  axis.title.x = element_text(color = "black", size = 14, margin = margin(t = 10)),
  axis.title.y.right = element_text(color = "black", size = 14, angle = 90, margin =
margin(l = 15)),
  plot.margin = margin(l = 5, r = 5, t = 5, b = 5))
```

Specific label configurations for precise placement

```
labs_base <- labs(x = NULL, y = NULL, title = NULL)
labs_x_only <- labs(x = "Time (weeks)", y = NULL, title = NULL)
labs_y_only <- labs(x = NULL, y = "Cumulative incidence\nof tumor development", title =
NULL)
labs_xy <- labs(x = "Time (weeks)", y = "Cumulative incidence\nof tumor development",
title = NULL)
```

10.2 Panel A: by sleep condition

Cumulative incidence of tumor onset by sleep condition.

```
A <- cuminc(Surv(age_followup, factor(status_competing)) ~ sleep_condition,
  data = tum_df) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
  labs_xy + scale_y + sleep_scale + scale_x +
  theme_bw() + theme_common + show_y +
  theme(legend.position = "none") +
  plot_annotation(title = "Sleep condition effect",
    theme = theme(plot.title = element_text(
      hjust = 0.5, face = "bold", size = 16, margin = margin(b = 10))))
```

Extract legend from plot A to use later on the full figure.

```
shared_legend <- cowplot::get_legend(
  A + theme(legend.position = "bottom",
    legend.direction = "horizontal",
    legend.text = element_text(size = 13),
    legend.title = element_text(size = 14, face = "bold")))
```

10.3 Panel B: by sleep condition for each lineage

10.3.1 HO_MT

Cumulative incidence of tumor onset by sleep condition for the lineage HO_MT.

```
B1 <- cuminc(Surv(age_followup, factor(status_competing)) ~ sleep_condition,
  data = tum_df %>% filter(lineage == "HO_MT")) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
```

```
labs_base + scale_y + scale_x + sleep_scale +
theme_bw() + theme_common + hide_y +
theme(legend.position = "none") +
annotate("text", x = 0.2, y = 0.95, label = "HO_MT",
          color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

10.3.2 HO_SPC

Same method as above, but for the lineage HO_SPC.

```
B2 <- cuminc(Surv(age_followup, factor(status_competing)) ~ sleep_condition,
             data = tum_df %>% filter(lineage == "HO_SPC")) %>%
ggcuminc(outcome = "1") +
add_confidence_interval() +
labs_x_only + scale_y + scale_x + sleep_scale +
theme_bw() + theme_common + hide_y +
theme(legend.position = "none") +
annotate("text", x = 0.2, y = 0.95, label = "HO_SPC",
          color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

10.3.3 HO_SPT

Same method as above, but for the lineage HO_SPT.

```
B3 <- cuminc(Surv(age_followup, factor(status_competing)) ~ sleep_condition,
             data = tum_df %>% filter(lineage == "HO_SPT")) %>%
ggcuminc(outcome = "1") +
add_confidence_interval() +
labs_y_only + scale_y + scale_x + sleep_scale +
theme_bw() + theme_common + show_y +
theme(legend.position = "none") +
annotate("text", x = 0.2, y = 0.95, label = "HO_SPT",
          color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

Create panel B combining plots B1, B2, and B3

```
B <- (B1 + B2 + B3 + plot_layout(nrow = 1)) +
plot_annotation(title = "Interaction between sleep condition and lineage",
                theme = theme(plot.title = element_text(hjust = 0.5, face = "bold",
                                                         size = 16, margin = margin(b =
                                                         10))))
```

10.4 Panel C: by lineage

10.4.1 HO_MT

Cumulative incidence of tumor onset for the lineage HO_MT.

```
C1 <- cuminc(Surv(age_followup, factor(status_competing)) ~ 1,
             data = tum_df %>% filter(lineage == "HO_MT")) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
  labs_base + scale_y + scale_x +
  theme_bw() + theme_common + hide_y +
  annotate("text", x = 0.2, y = 0.95, label = "HO_MT",
           color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

10.4.2 HO_SPC

Same method as above, but for the lineage HO_SPC.

```
C2 <- cuminc(Surv(age_followup, factor(status_competing)) ~ 1,
             data = tum_df %>% filter(lineage == "HO_SPC")) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
  labs_x_only + scale_y + scale_x +
  theme_bw() + theme_common + hide_y +
  annotate("text", x = 0.2, y = 0.95, label = "HO_SPC",
           color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

10.4.3 HO_SPT

Same method as above, but for the lineage HO_SPT.

```
C3 <- cuminc(Surv(age_followup, factor(status_competing)) ~ 1,
             data = tum_df %>% filter(lineage == "HO_SPT")) %>%
  ggcuminc(outcome = "1") +
  add_confidence_interval() +
  labs_y_only + scale_y + scale_x +
  theme_bw() + theme_common + show_y +
  annotate("text", x = 0.2, y = 0.95, label = "HO_SPT",
           color = "black", fontface = "bold", size = 5, hjust = 0, vjust = 1)
```

Create panel C combining plots C1, C2, and C3

```
C <- (C1 + C2 + C3 + plot_layout(nrow = 1)) +
  plot_annotation(title = "Lineage effect",
                  theme = theme(plot.title = element_text(
                    hjust = 0.5, face = "bold", size = 16, margin = margin(b = 10))))
```

10.5 Assembly

Add a letter tag (A/B/C) to the first sub-plot of each panel.

```
A <- A + labs(tag = "A") + theme(plot.tag = element_text(size = 20, face = "bold"))
B[[1]] <- B[[1]] + labs(tag = "B") + theme(plot.tag = element_text(size = 20, face =
"bold"))
C[[1]] <- C[[1]] + labs(tag = "C") + theme(plot.tag = element_text(size = 20, face =
"bold"))
```

Convert each panel into a graphical object. (Wrapping A in patchwork to force perfect alignment with the B and C grids).

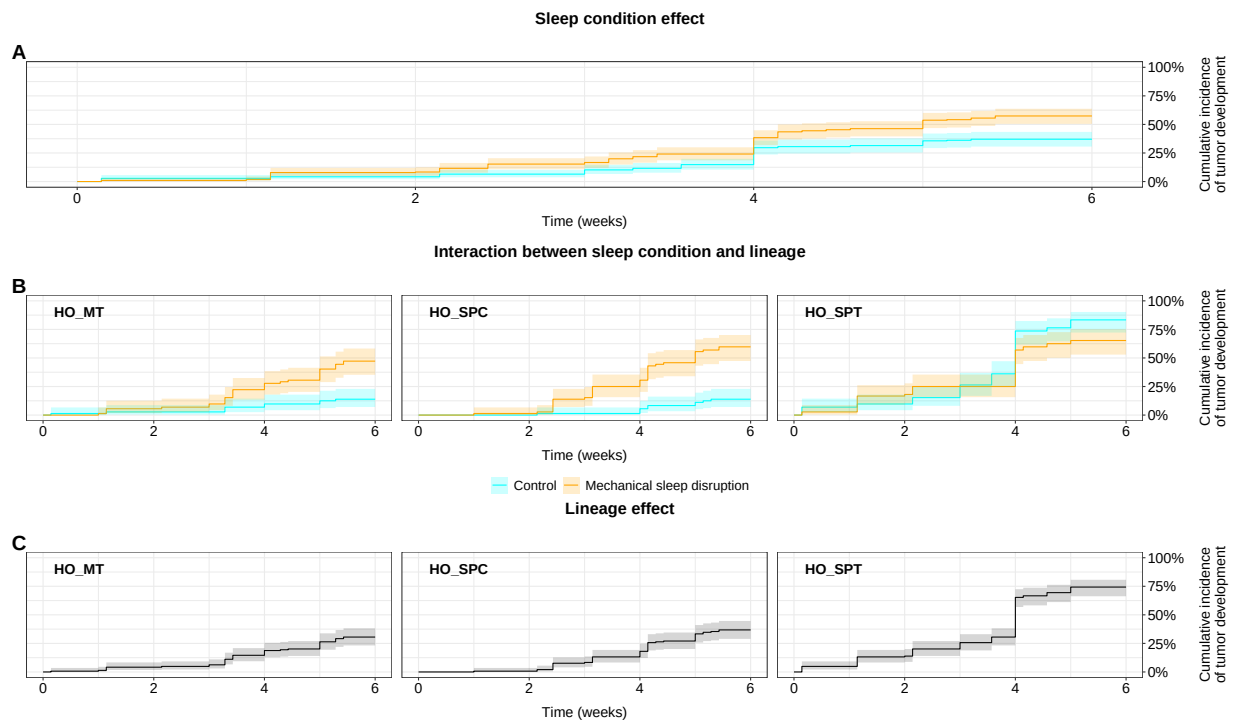
```
A <- patchwork::patchworkGrob(A + patchwork::plot_layout(ncol = 1))
B <- patchwork::patchworkGrob(B)
C <- patchwork::patchworkGrob(C)
```

Combine the panels and the shared legend vertically. The legend is placed between B and C.

```
final_tum_panel <- cowplot::plot_grid(A,
  B,
  shared_legend,
  C,
  ncol = 1,
  align = "v",
  axis = "lr",
  rel_heights = c(1, 1, 0.1, 1)) +
  theme(plot.margin = margin(l = 5, r = 5, t = 5, b = 5))
```

Show the final figure.

```
print(final_tum_panel)
```



```
ggsave("figure_2.png", plot = final_tum_panel, width = 21, height = 29.7, units = "cm",
  dpi = 300)
```